

## • Quais são as entidades (classes) que devem ser criadas?

O domínio do e-commerce exige a separação entre o ciclo de seleção de produtos, o documento fiscal de fechamento e as regras de logística:

1. **Produto (Entidade)**: Classe que representa as mercadorias à venda, armazenando dados como identificador único, nome, preço unitário e o peso em quilogramas (essencial para o cálculo de frete).
2. **ItemPedido (Objeto de Valor)**: Classe que encapsula o vínculo de um **Produto** específico com a sua respectiva **Quantidade** comprada, sendo responsável por calcular seu próprio subtotal.
3. **Carrinho (Entidade de Suporte)**: Classe temporária que serve para o usuário acumular seus itens de interesse antes de decidir fechar a compra.
4. **Pedido (Entidade / Raiz de Agregação)**: A estrutura definitiva e oficial do negócio. Ela registra os itens comprados, o valor final dos produtos, o preço do frete, as informações de prazo e gerencia as transições de pagamento.
5. **ITipoEntrega (Interface/Abstração)**: Contrato que serve de base para as estratégias de envio. Dela derivam as classes concretas **EntregaNormal** e **EntregaExpressa**.
6. **MetodoPagamento (Enum)**: Enumerador que restringe as formas de quitação do pedido: **Cartao**, **Pix** e **Boleto**.
7. **StatusPedido (Enum)**: Enumerador para controlar os estados do fluxo da compra: **AguardandoPagamento**, **Pago** e **Cancelado**.

## • Quais dessas entidades possuem ações (métodos) e quais são eles?

Os comportamentos foram desenhados para que cada classe proteja a integridade de seus dados e execute seus cálculos internos:

1. **Na classe ItemPedido:**
  - **CalcularSubtotal()**: Multiplica o preço de venda do produto pela quantidade selecionada.
2. **Na classe Carrinho:**
  - **AdicionarProduto(Produto produto, int quantidade)**: Insere itens na lista ou incrementa a quantidade caso o produto já tenha sido adicionado anteriormente.
  - **CalcularTotalProdutos()** e **CalcularPesoTotal()**: Métodos de agregação que somam, respectivamente, o valor financeiro e o peso físico de todos os itens da lista.
  - **Limpar()**: Remove todos os itens da memória.
3. **Na classe Pedido:**
  - **ConfirmarPagamento(MetodoPagamento metodo)**: Valida se o pedido está apto a receber a quitação e altera seu status para "Pago".
4. **Nas classes de frete (EntregaNormal e EntregaExpressa):**
  - **CalcularFrete(decimal pesoTotal)** e **CalcularPrazoDias()**: Métodos que aplicam fórmulas distintas com base no peso total da mercadoria para retornar o valor monetário do transporte e o tempo estimado de entrega.

- **Há necessidade de criação de interface? Se sim, descreva os comportamentos que devem ser criados.**

Sim, a criação da interface **ITipoEntrega** é fundamental para atender à regra de que "cada tipo de entrega calcula o valor do frete e o prazo de forma diferente". A utilização desse padrão de projeto (*Strategy*) garante que o **Pedido** não fique amarrado a regras rígidas de correios ou transportadoras privadas. Se a empresa adotar uma nova modalidade (como "Retirada em Loja" ou "Entrega por Drone"), basta codificar uma nova classe sob essa interface.

- *Comportamentos/Contratos exigidos:*
  - Propriedade **string Nome { get; }** para exibir a modalidade ao cliente.
  - Método **decimal CalcularFrete(decimal pesoTotal);** para computar a taxa de envio.
  - Método **int CalcularPrazoDias();** para fornecer o tempo estimado de trânsito em dias úteis.

- **Quais são os relacionamentos entre as classes e entidades criadas?**

O sistema conecta-se através de relações de agregação de itens e dependências flexíveis por interfaces:

1. **Composição e Associação de Itens (\$1:N\$):** Tanto o **Carrinho** quanto o **Pedido** possuem uma lista interna (**List<ItemPedido>**). O **ItemPedido**, por sua vez, aponta para um único **Produto** (\$1:1\$).
2. **Dependência via Injeção por Interface:** O construtor da classe **Pedido** recebe e armazena uma referência do tipo **ITipoEntrega**. O pedido interage com essa interface de maneira genérica sem precisar saber a lógica interna usada para despachar a encomenda.
3. **Fluxo de Conversão Dinâmica:** Existe uma relação de transição onde o **Pedido** consome os dados finais de um **Carrinho** para preencher suas propriedades e, ao concluir o processo, aciona o método de limpeza do carrinho.

- **Onde o desenvolvedor deve tratar exceções em caso de críticas no processo?**

O gerenciamento de erros atua impedindo comportamentos inconsistentes no carrinho e fraudes de fluxo no faturamento:

1. **Na Camada de Domínio (Dentro das Regras de Negócio):** O desenvolvedor deve lançar exceções do tipo **ArgumentException** ou **InvalidOperationException** imediatamente para travar o código nas seguintes críticas:
  - No construtor de **ItemPedido**, se a quantidade informada for igual ou menor que zero.
  - No construtor de **Pedido**, se houver uma tentativa de fechar uma venda utilizando um carrinho que esteja totalmente vazio.
  - No método **ConfirmarPagamento**, caso o operador ou o sistema tentem marcar como pago um pedido que já foi cancelado ou que já havia sido liquidado anteriormente.

2. **Na Camada de Aplicação / Controladores da API:** O desenvolvedor deve implementar estruturas **try-catch** ao realizar a conversão do carrinho e ao processar a resposta dos gateways de pagamento. Se o cartão de crédito do usuário for recusado pelo banco, ou se o boleto bancário expirar, a camada de aplicação captura essa falha, registra o log técnico do erro com o código da operadora para fins de auditoria, e converte essa resposta em uma instrução clara para a tela do e-commerce, como: *"Transação recusada: Por favor, verifique o saldo ou os dados digitados do seu cartão."*